

```
    } else {  
        // RECURSIVE CASE  
        return factorial(number - 1, accum * number);  
    }  
}  
  
document.write(factorial(5));  
</script>
```

Notice that the `factorial()` function's recursive case ends with a `return` statement returning the results of a recursive call to `factorial()`. To allow the interpreter or compiler to implement tail call optimization, the last action a recursive function makes must be to return the results of the recursive call. No instructions can occur between making the recursive call and the `return` statement. The base case returns the `accum` parameter. This is the accumulator, explained in the next section.

To understand how tail call optimization works, remember from Chapter 1 what happens when a function is called. First, a frame object is created and stored on the call stack. If the function call calls another function, another frame object is created and placed on top of the first frame object on the call stack. When a function returns, your program automatically deletes the frame object from the top of the stack.

A stack overflow happens when too many function calls are made without returning, causing the number of frame objects to exceed the capacity of the call stack. This capacity is 1,000 function calls for Python and about 10,000 for JavaScript programs. While these amounts are more than enough for typical programs, recursive algorithms could exceed this limit and cause a stack overflow that crashes your program.

Recall from Chapter 2 that a frame object stores the local variables in the function call as well as the return address of the instruction to return to when the function finishes. However, if the last action in the recursive case of a function is to return the results of a recursive function call, there's no need to retain the local variables. The function does nothing involving the local variables after the recursive call, so the current frame object can be deleted immediately. The next frame object's return address information can be the same as the old deleted frame object's return address.